



INSTITUTO FEDERAL
Rio Grande do Sul

Comparação entre Modelagem Relacional e Objeto-Relacional

Domínio: Spotify Tracks
Dataset

Mateus Medeiros Schneider
Miguel Bohrz Vogel



Domínio da aplicação

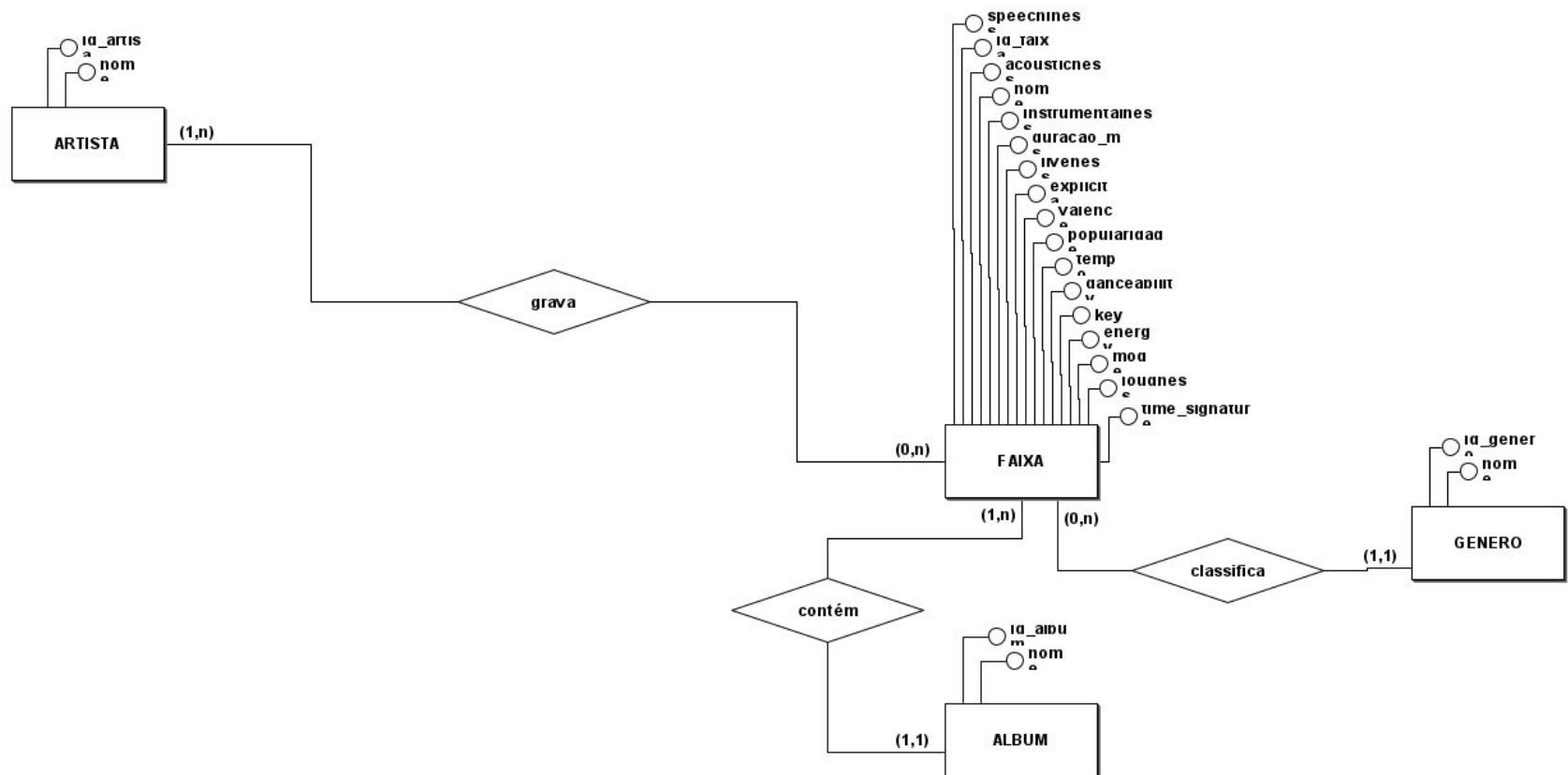
Dataset público do Kaggle (Spotify Tracks Dataset) com ~114 mil faixas

Cada registro traz: artista(s), álbum, gênero, popularidade e características de áudio (danceability, energy, tempo, etc.)

Domínio rico o suficiente pra explorar atributos multivalorados, tipos compostos e regras de domínio



Modelo Conceitual (MER)



Esquema Relacional (I)

artista(id_artista, nome)

album(id_album, nome)

genero(id_genero, nome)

faixa(id_faixa, nome, duracao_ms, explicita, popularidade,
danceability, energy, loudness, speechiness, acousticness,
instrumentalness, liveness, valence, tempo, key, mode, time_signature,
id_album#, id_genero#)

(id_album) referencia album(id_album)

(id_genero) referencia genero(id_genero)

faixa_artista(id_faixa#, id_artista#)

(id_faixa) referencia faixa(id_faixa)

(id_artista) referencia artista(id_artista)



Esquema Objeto-Relacional

album(id_album, nome)

genero(id_genero, nome)

faixa(id_faixa, nome, duracao_ms, explicita, popularidade,
features, nota_chave, mode, time_signature,
artistas[], id_album#, id_genero#)

(id_album) referencia album(id_album)

(id_genero) referencia genero(id_genero)



SQL Relacional

6 tabelas no total (`artista`, `album`, `genero`, `faixa`, `faixa_artista`)

Tipos primitivos padrão (`INTEGER`, `VARCHAR`, `NUMERIC`, `BOOLEAN`)

Regras de validação via `CHECK` espalhadas tabela por tabela (ex.: popularidade, duração, key)

Relacionamento N:N (artista–faixa) exige tabela extra só pra existir

SQL 100% portátil — roda em qualquer SGBD relacional (MySQL, SQL Server, etc.), sem depender de recursos exclusivos do PostgreSQL.



SQL Relacional - Comandos

```
CREATE TABLE faixa (  
    id_faixa          VARCHAR(30) PRIMARY KEY,  
    nome              VARCHAR(300) NOT NULL,  
    duracao_ms        INTEGER NOT NULL CHECK (duracao_ms > 0),  
    explicita         BOOLEAN NOT NULL DEFAULT FALSE,  
    popularidade      INTEGER NOT NULL CHECK (popularidade BETWEEN 0 AND 100),  
    nota_chave        SMALLINT CHECK (nota_chave BETWEEN -1 AND 11),  
    ...  
    id_album          INTEGER NOT NULL REFERENCES album(id_album),  
    id_genero          INTEGER NOT NULL REFERENCES genero(id_genero)  
);
```



Objeto-Relacional: visão geral

ENUM (nota musical)

DOMAIN (popularidade)

COMPOSITE TYPE (audio features)

ARRAY (artistas)



Recurso 1: ENUM

```
CREATE TYPE nota_musical AS ENUM (  
  'C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#',  
  'B', 'INDEFINIDA'  
);
```

Substitui o código numérico (0-11) do Spotify por valores nomeados, com validação automática de valores inválidos.



Recurso 2: DOMAIN

```
CREATE DOMAIN popularidade_spotify AS  
INTEGER  
CHECK (VALUE BETWEEN 0 AND 100);
```

Centraliza a regra "0 a 100" num tipo reaproveitável, evitando repetir CHECK em cada tabela que precisar de popularidade.



Recurso 3: COMPOSITE TYPE

```
CREATE TYPE audio_features AS (  
    danceability NUMERIC(5,4), energy NUMERIC(5,4),  
    loudness NUMERIC(6,3), ... tempo NUMERIC(6,2)  
);
```

Agrupa os 9 atributos de áudio como uma unidade lógica só, refletindo que essas métricas nascem juntas na análise do Spotify.



Recurso 4: ARRAY

artistas VARCHAR(200)[] NOT NULL

Elimina a tabela associativa faixa_artista. Ganha simplicidade de escrita, perde a capacidade de guardar atributos do relacionamento (ex.: papel do artista).



Herança

```
CREATE TABLE faixa_explicita (  
    motivo_flag VARCHAR(100)  
) INHERITS (faixa);
```

Caso mais fraco dos recursos: só 1 atributo extra (motivo_flag) não justifica muito a complexidade herdada: usado aqui como contraste na comparação.



Exemplos de uso comparados

Relacional: buscar faixas de um artista exige JOIN com `faixa_artista`

OR: mesma busca é direta com `ANY(artistas)`, sem JOIN

Relacional: nota musical é `SMALLINT`, sem validação semântica

OR: `ENUM` garante e documenta os valores possíveis



Tabela comparativa

Critério	Relacional	Objeto-Relacional
Legibilidade	Média (códigos numéricos)	Alta (tipos nomeados)
Redundância/Norm alização	Alta normalização, mais tabelas	Menos tabelas, dados agrupados
Complexidade de query	Mais JOINS	Queries mais diretas
Integridade de dados	Via CHECK espalhado	Via DOMAIN/ENUM centralizados
Portabilidade	Alta (SQL padrão)	Baixa (específico do PostgreSQL)



Quando usar cada um

Relacional: quando portabilidade e simplicidade importam mais (multi-SGBD)

Objeto-Relacional: quando o domínio tem estrutura naturalmente aninhada, multivalorada ou com regras de domínio recorrentes

Spotify Tracks se encaixa bem no segundo caso (artistas múltiplos, features agrupadas)



Considerações finais

Array e Composite Type foram os recursos que mais simplificaram o schema

Herança foi o mais fraco: domínio não tem uma hierarquia de subtipos rica

Trade-off central: OR ganha expressividade e perde portabilidade/padronização

